

# Security

Who you are, what that gets you, and what actually protects the database.

---

The identity model in SD Core is not OpenQM's and not SD on Linux's. It is worth understanding before you test anything else, because several behaviours that look like bugs are consequences of it.

---

## Signing in asks for no password

**The operating system has already authenticated you. SD asks Windows who you are.**

|                                    |                                                                                                                                                                                      |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sd</code> , no account named | you land in <b>the SD account with your own name</b>                                                                                                                                 |
| no SD account of that name         | refused — <i>Account %1 not in register</i> (5018)                                                                                                                                   |
| not in <code>sdusers</code>        | refused at the door — <i>not registered for SD use</i> (5009)                                                                                                                        |
| <code>sd -A&lt;name&gt;</code>     | <b>refused unless <code>&lt;name&gt;</code> is your own account</b> (10051)                                                                                                          |
| an elevated session                | <b>your own account, like everybody else.</b> <code>logto sdsys</code> afterwards                                                                                                    |
| <code>logto sdsys</code>           | asks for no password. <b>The gate is elevation</b> — a UAC consent prompt if the session is not already elevated, and 10002 with an audited <code>LOGTO REFUSED</code> if that fails |

**There is no SDSYS password.** There is deliberately no second shared secret held by every administrator — that is the OpenQM weakness this exists to remove.

`sd <command>` **never asks you to set a password.** It runs and exits, so it is not a session anybody is being invited into. In earlier builds of this port it walked into a prompt it could never be given input for. See Running SD.

---

## Being an administrator

**If you can log in to Windows as an administrator, you are an administrator of sd.** The person who installs SD is an SD administrator without any further step.

Two different questions are asked in two different places, and both are wanted:

| QUESTION                         | HOW IT IS ANSWERED              | GATES                                                                                                         |
|----------------------------------|---------------------------------|---------------------------------------------------------------------------------------------------------------|
| <i>Are you an administrator?</i> | the account's groups in the SAM | <code>sd -start</code> — starting the server should not demand elevation of somebody already an administrator |
| <i>Are you elevated?</i>         | the process token               | <b>reaching SDSYS</b> , and every privileged action                                                           |

A UAC-filtered token carries `Administrators` as *deny only*, so these give different answers for the same person, and conflating them is the easy mistake.

**A third question is asked as well: *where did this session come from?*** Being an administrator and being elevated are no longer enough on their own — the session also has to have started on this computer. An administrator signing in from another machine is refused, over ssh and over the API alike, whichever way the first two questions are answered.

**One property to accept consciously.** `Administrators` is machine-wide, so anyone in it for an unrelated reason — the machine's own administrator, a domain admin, an IT tool's service account — gets SDSYS. Linux sudoers is machine-wide too, so this is parity rather than a Windows weakness, but it should be a decision rather than a discovery.

## Taking an account out of use without deleting it

`modify.account fred suspended` denies entry at all three ways in — ssh and the console, `logto`, and the API. It is reversible with nothing to remember: the VOC and every Windows group membership are left exactly as they are, and the tier it displaced is recorded, so naming a tier brings the account back where it was.

**Understand what it is and is not, because the name oversells it.**

|              |                                                 |
|--------------|-------------------------------------------------|
| <b>It is</b> | an SD control. The three doors SD owns are shut |
|--------------|-------------------------------------------------|

|                  |                                               |
|------------------|-----------------------------------------------|
| <b>It is not</b> | a Windows control. Nothing is withdrawn there |
|------------------|-----------------------------------------------|

So a suspended user's ssh connection is still accepted and SD still starts before refusing them, and **a suspended administrator keeps `Administrators`, keeps their `os.users` record, and can still elevate on this machine.** An elevated session can also still `logto` into a suspended account, which is deliberate — that is how you look at one.

**If you are Suspending an account to contain somebody rather than to park it, disable the Windows account too.** Everything on this page rests on Windows identity; a control that does not touch Windows cannot be the whole answer.

## Understand what the security position rests on

**NOTHING IN SD CHECKS A SECRET AT CONSOLE OR ssh LOGIN. ACCESS IS ENTIRELY OPERATING-SYSTEM GROUP MEMBERSHIP.**

That is not a weakening. Every SD process opens the database directly, in your own process, under your own token. There is no data server standing between you and the files. So:

**While SD runs as the invoking user, account passwords organise access; they do not secure it.**

A password gate inside SD is not a file security boundary. The old password model implied one the filesystem never enforced. This states the real position instead of dressing it up.

**Passwords still matter for the API**, which is a separate door and does require one — see API access.

## What actually protects the database

### THE TREE IS PRIVATE FROM THE REST OF THE MACHINE

`C:\ProgramData` grants `BUILTIN\Users` read and execute by inheritance, so the Windows default is world-readable and snooping needs no privilege at all. The installer **breaks inheritance first** and grants narrowly: `SYSTEM`, `Administrators` and `sdusers`.

**This is the step that makes the data private, and nothing at run time Substitutes for it.**

### YOUR ACCOUNT DIRECTORY IS LOCKED TO YOU

`C:\ProgramData\SD\user_accounts\<name>` is granted to that account's own group, to administrators and to the system, **and to nobody else**.

Before this, every SD user could read and rewrite every other user's account files from Explorer or a command prompt — outside SD, so none of SD's own permission checks applied. SD has always refused to let you `logto` an account you are not a member of; this makes the files agree with that.

**Nothing you type changes, and your own account is unaffected** — you are a member of its group.

A directory whose Windows group has been removed is **skipped deliberately** and named in the report. Locking it to a group that no longer exists would take the account away from the person who owns it.

### THE SYSTEM DIRECTORIES ARE READ-ONLY TO ORDINARY USERS

Seven parts of the SD data area could be changed by anyone in `sdusers`. They are now readable by everyone who needs them and writable only by an administrator:

|                             |                                    |                           |
|-----------------------------|------------------------------------|---------------------------|
| <code>sdsys\accounts</code> | the register of accounts           |                           |
| <code>sdsys\map</code>      | <code>sdsys\messages</code>        | <code>sdsys\newvoc</code> |
| <code>sdsys\bp</code>       | <code>sdsys\cat</code>             | SD's own programs         |
| <code>sd.conf</code>        | the configuration read at start-up |                           |

**Nothing an ordinary user does needs to write them** — not the spooler, not saved lists, not a phantom. The commands that *do* write them are already administrator commands and are unaffected.

`sdsys\%ipc` is **deliberately unchanged**. Every session writes to it, and it is how a `phantom` is given its command.

The global catalogue and the pcode library are locked the same way.

**An existing installation is not changed by an upgrade**, because the installer never overwrites a data area that is already there. To apply it to one you already have, run this once per path from an elevated prompt:

```
powershell -File "C:\Program Files\SD\secure-sysdirs.ps1" -Path "C:\ProgramData\SD\sdsys\accounts"
```

## THE CREDENTIAL FILE

**ON INSTALLS MADE BEFORE 17 Aug 2026, EVERY SD USER COULD WRITE THE FILE SD KEEPS ACCOUNT PASSWORDS IN.** No password is stored there — SD keeps a scrambled verifier that cannot be turned back into a password — but **being able to replace one was enough**: a user could put in a verifier for a password of their own choosing and sign in as somebody else, including through the API.

The installer was always meant to lock that file down and never did. The step ran, reported nothing, and had no effect, because of a quoting fault in how it was called. **Nothing in the install looked wrong.**

**Reinstalling is what fixes it.** To check without reinstalling, from an **ordinary** (not administrator) prompt:

```
icaccls "C:\ProgramData\SD\sdsys\%CRED"
```

If that prints permissions, the file is still open. **If it says "Access is denied", it is protected — which is what you want.**

---

## Continued in

Security and the operating system — reaching the operating system from inside SD, privileged work, and the audit trail.